

CS 305: Computer Networks

Fall 2025

Lecture 11: Network Layer – The Data Plane

Ming Tang

Department of Computer Science and Engineering
Southern University of Science and Technology (SUSTech)

Chapter 4: outline

4.1 Overview of Network layer

- data plane
- control plane

4.2 What's inside a router

4.3 IP: Internet Protocol

- datagram format
- fragmentation
- IPv4 addressing
- network address translation
- IPv6

4.4 Generalized Forward and SDN

- match
- action
- OpenFlow examples of match-plus-action in action

NAT: network address translation

Local area networks (LANs):

- E.g., a residence, school, laboratory, university campus or office building
- a range of addresses would need to be allocated by the ISP to cover all of the LAN's IP devices
- If the subnet grew bigger, a larger block of addresses would have to be allocated.
- Huge number of LAN ...

不要让局域网 (LAN) 里的设备直接暴露在公网上，而是给它们分配“内部专用”的地址。

NAT: network address translation

NAT: reserve blocks of IP addresses for LANs

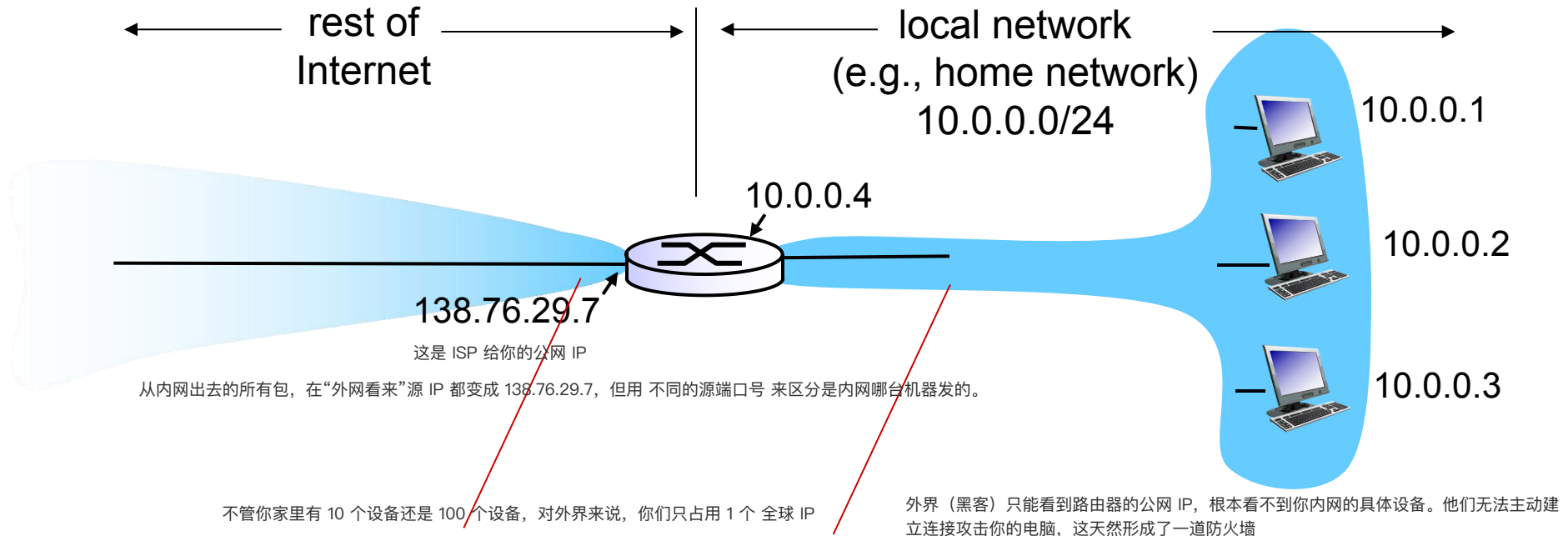
- Private network; private IP address

RFC 1918 name	IP address range	Number of addresses	Largest CIDR block (subnet mask)	Host ID size	Mask bits	<i>Classful</i> description ^[Note 1]
24-bit block	10.0.0.0 – 10.255.255.255	16 777 216	10.0.0.0/8 (255.0.0.0)	24 bits	8 bits	single class A network
20-bit block	172.16.0.0 – 172.31.255.255	1 048 576	172.16.0.0/12 (255.240.0.0)	20 bits	12 bits	16 contiguous class B networks
16-bit block	192.168.0.0 – 192.168.255.255	65 536	192.168.0.0/16 (255.255.0.0)	16 bits	16 bits	256 contiguous class C networks

以上这3类都是给局域网内部用的私有ip地址段，在公网中不会被路由转发

NAT (Network Address Translation , 网络地址转换) : 让一整个局域网 (LAN) 里的很多设备 , 用一个公网 IP 去上网 ; 对外看起来“都像同一台机器”。

NAT: network address translation



all datagrams *leaving* local network have *same* single source NAT IP address: 138.76.29.7, different source port numbers

datagrams with source or destination in this network have 10.0.0/24 address for source, destination (as usual)

NAT: network address translation

motivation: local network uses just one IP address (e.g., 138.76.29.7 in the example) as far as outside world is concerned:

- just one IP address for all devices
- can change addresses of devices in local network without notifying outside world
内网怎么折腾，外网都不知道
- can change ISP without changing addresses of devices in local network
换 ISP，不用重配全家设
- devices inside local network not explicitly addressable, visible by outside world (a security plus)

内网设备 不能被外网直接用 IP 定位到 / 看见

这带来一个“副作用式的安全好处”:

外网不能随便扫描、主动连接你内网设备

相当于一个 天然的“默认拒绝”防火墙

NAT ≠ 防火墙

只是“顺带隐藏”，真正的安全还需要防火墙规则

NAT: network address translation

implementation: NAT router must:

- *outgoing datagrams: replace* (source IP address, port #) of every outgoing datagram to (NAT IP address, new port #) . . . remote clients/servers **will respond using (NAT IP address, new port #)** as destination addr
- *remember (in NAT translation table)* every (source IP address, port #) to (NAT IP address, new port #) translation pair
- *incoming datagrams: replace* (NAT IP address, new port #) in **dest fields** of every incoming datagram with corresponding (source IP address, port #) stored in NAT table

为什么要改端口?

因为

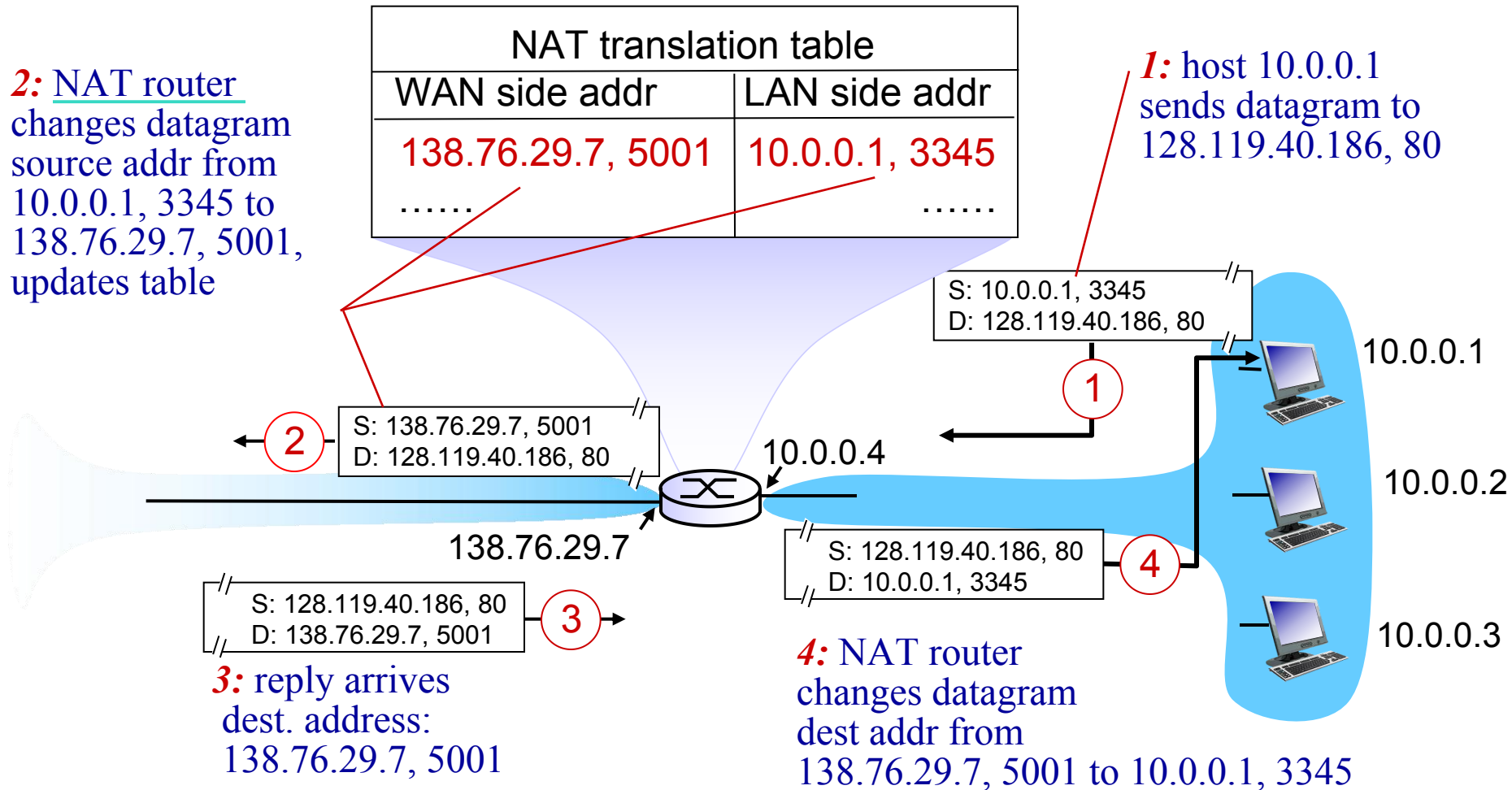
多台内网主机

都共用同一个公网 IP

只能靠端口号区分是谁

这叫 PAT (Port Address Translation)

NAT: network address translation



* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

NAT: network address translation

NAT是靠修改端口号来区分内网不同设备的

- 16-bit port-number field:
 - 60,000 simultaneous connections with a single LAN-side address!
- NAT is controversial:
 - routers should only process up to layer 3
 - violates end-to-end argument
 - NAT possibility must be taken into account by app designers, e.g., P2P applications, server processes
 - Well-known port number

原则：根据 OSI 模型，路由器是 网络层 (Layer 3) 设备，它应该只看 IP 地址。

NAT 的越界：NAT 路由器为了工作，必须拆开数据包，去修改 传输层 (Layer 4) 的端口号。

原则：互联网的设计初衷是，任何两台主机应该能直接对话，中间的网络只负责传数据，不应该修改数据。

NAT 的破坏：NAT 强行插入中间，修改了源地址和端口。外网主机看到的不是真实的你，而是路由器的“马甲”。

这会导致很多应用层的问题

Chapter 4: outline

4.1 Overview of Network layer

- data plane
- control plane

4.2 What's inside a router

4.3 IP: Internet Protocol

- datagram format
- fragmentation
- IPv4 addressing
- network address translation
- **IPv6**

4.4 Generalized Forward and SDN

- match
- action
- OpenFlow examples of match-plus-action in action

IPv6: motivation

- *initial motivation*: 32-bit address space soon to be completely allocated.
- additional motivation:
 - **header format** helps to speed up processing/forwarding

ipv4快用完了，并且ipv4header太复杂了，路由器处理起来很慢

IPv6 datagram format:

- 128-bit address space: why? 多到“可以随便分”
- **fixed-length 40 byte** header; why?
- no fragmentation allowed; why?

40bytes header好处:

硬件转发 (ASIC) 极其友好

无条件定长解析

pipeline 更简单

核心思想:

暴功能 = 可选扩展头 (extension headers)

暴转发 = 固定头

IPv4 :

- 路由器可以分片
- 问题:
 - 路由器负担重
 - 丢一个分片，整包作废
 - DoS 攻击温床

IPv6 :

- 路由器一律不分片
- 如果包太大:
 - 路由器直接丢弃
 - 返回 ICMPv6 : Packet Too Big
- 源主机自己调整 MTU

结果:

- 路由器更简单、更快
- 分片只发生在 端系统
- 网络更可预测

IPv6 : 路由器觉得包太大，直接丢弃，并回发一个“包太大 (Packet Too Big)”的报错信息给源主机。源主机自己要把包切小点再发。

IPv6 datagram format

priority: identify priority among datagrams in flow

flow Label: identify datagrams in same “flow.”
(concept of “flow” not well defined).

next header: identify upper layer protocol for data (for example, to TCP or UDP).

Flow Label 的目的：

让路由器 不解析 TCP/UDP

直接根据 (src, dst, flow label) 快速分类转发

现实情况：

标

准定义比较模糊

实

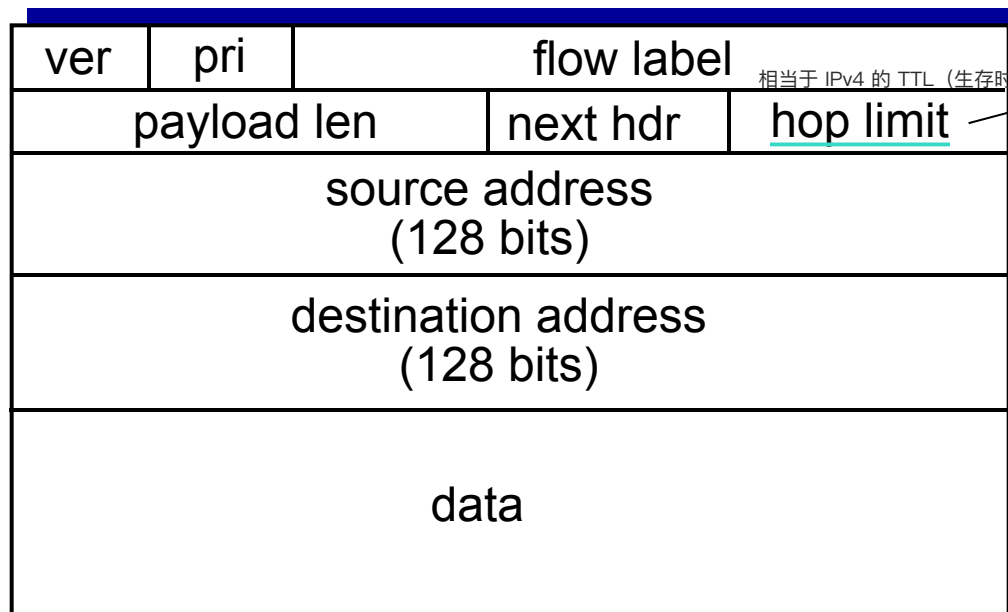
际部署中：

使

用不广泛

但

为未来高速网络 留了接口



相当于 IPv4 的 TTL (生存时间)，防止死循环。

decremented by one by each router that forwards the Datagram; if reaches zero, the datagram is discarded.

header 40bytes , 不包含data

← 32 bits →

Other changes from IPv4

每一跳路由器都要算校验和太费时间了。而且现在的链路层（光纤、以太网）和传输层（TCP/UDP）都有校验，网络层再做一次属于重复劳动。

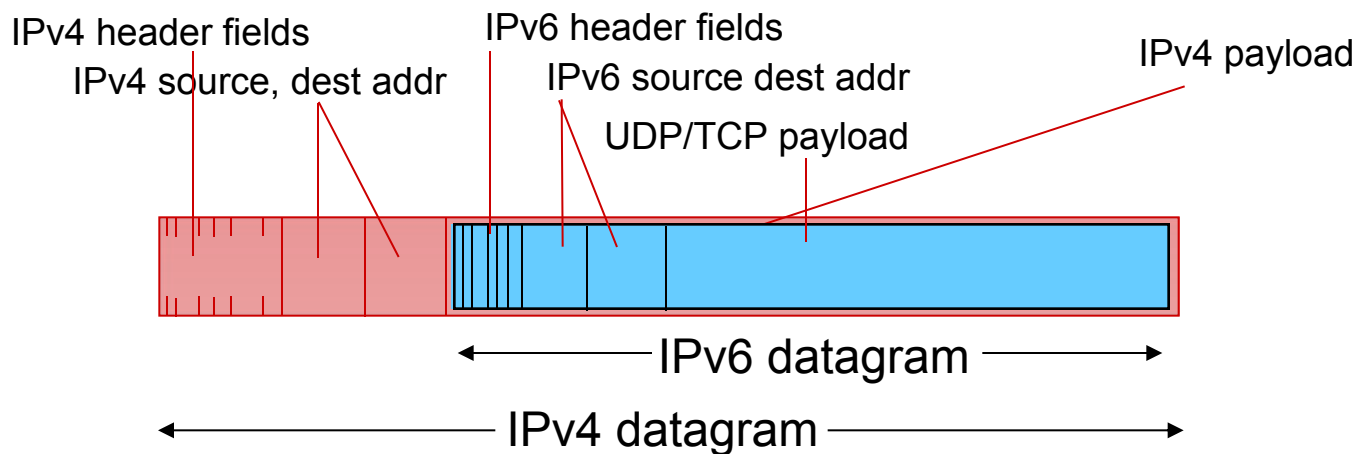
- *checksum*: removed entirely to reduce processing time at each hop; why?
- *options*: allowed, but outside of header, indicated by “Next Header” field
- *no fragmentation*:
 - too large to be forwarded over the outgoing link
 - the router simply drops the datagram and sends a “Packet Too Big” ICMP error message

IPv4 的选项字段让头部长度不固定。IPv6 把选项移到了“扩展头部”里（通过 Next Header 指针链式查找），保持了主头部的简洁。

Transition from IPv4 to IPv6

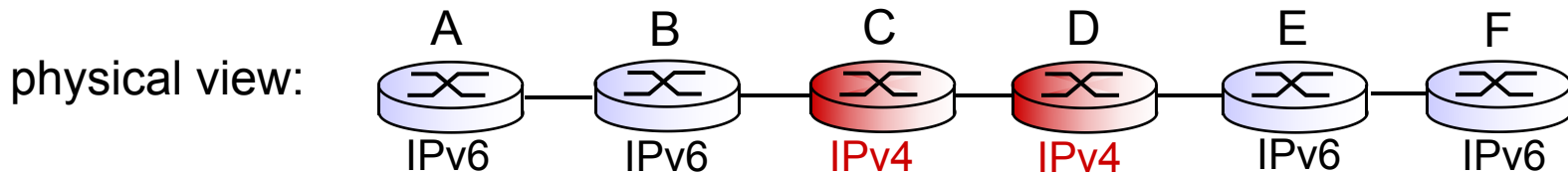
- not all routers can be upgraded simultaneously
 - no “flag days”
 - how will network operate with mixed IPv4 and IPv6 routers?
- **tunneling:** IPv6 datagram carried as *payload* in IPv4 datagram among IPv4 routers

把 IPv6 datagram 整个当成“数据载荷 payload”，外面再套一层 IPv4 的壳（IPv4 header）



Tunneling

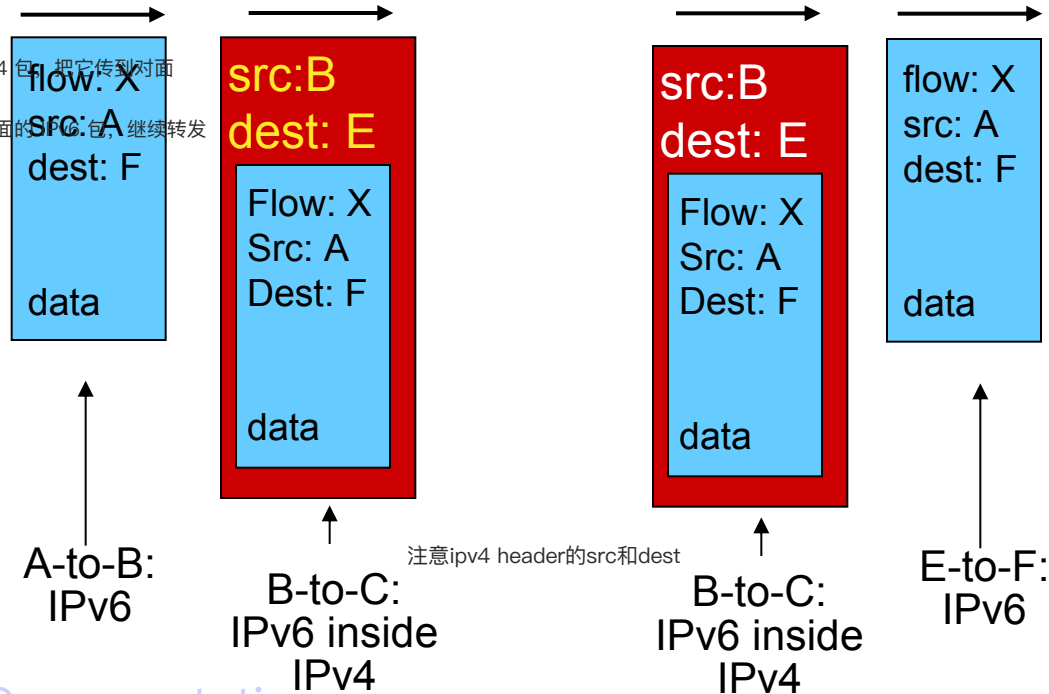
IPv6 的核心设计哲学是：简单的就是最快的。它把复杂的脏活累活（分片、校验）都甩给了端系统（电脑/手机），让中间的路由器只做一件事：极速转发。



边界路由器给它套上一个 IPv4 的头（封装）

中间的 IPv4 路由器只看外面的头，以为它是一个普通的 IPv4 包，把它传到对面

对面的边界路由器把 IPv4 头拆掉（解封装），取出里面的 IPv6 包，继续转发



注意ipv4 header的src和dest

IPv6: adoption

- Google: 8% of clients access services via IPv6
- NIST: 1/3 of all US government domains are IPv6 capable
- *Long (long!) time for deployment, use*
 - 20 years and counting!
 - think of application-level changes in last 20 years: WWW, Facebook, streaming media, Skype, ...
 - *Why?*

Chapter 4: outline

4.1 Overview of Network layer

- data plane
- control plane

4.2 What's inside a router

4.3 IP: Internet Protocol

- datagram format
- fragmentation
- IPv4 addressing
- network address translation
- IPv6

4.4 Generalized Forward and SDN

- match
- action
- OpenFlow examples of match-plus-action in action

Generalized Forwarding and SDN

Each router contains a **flow table** that is computed and distributed by a *logically centralized* routing controller

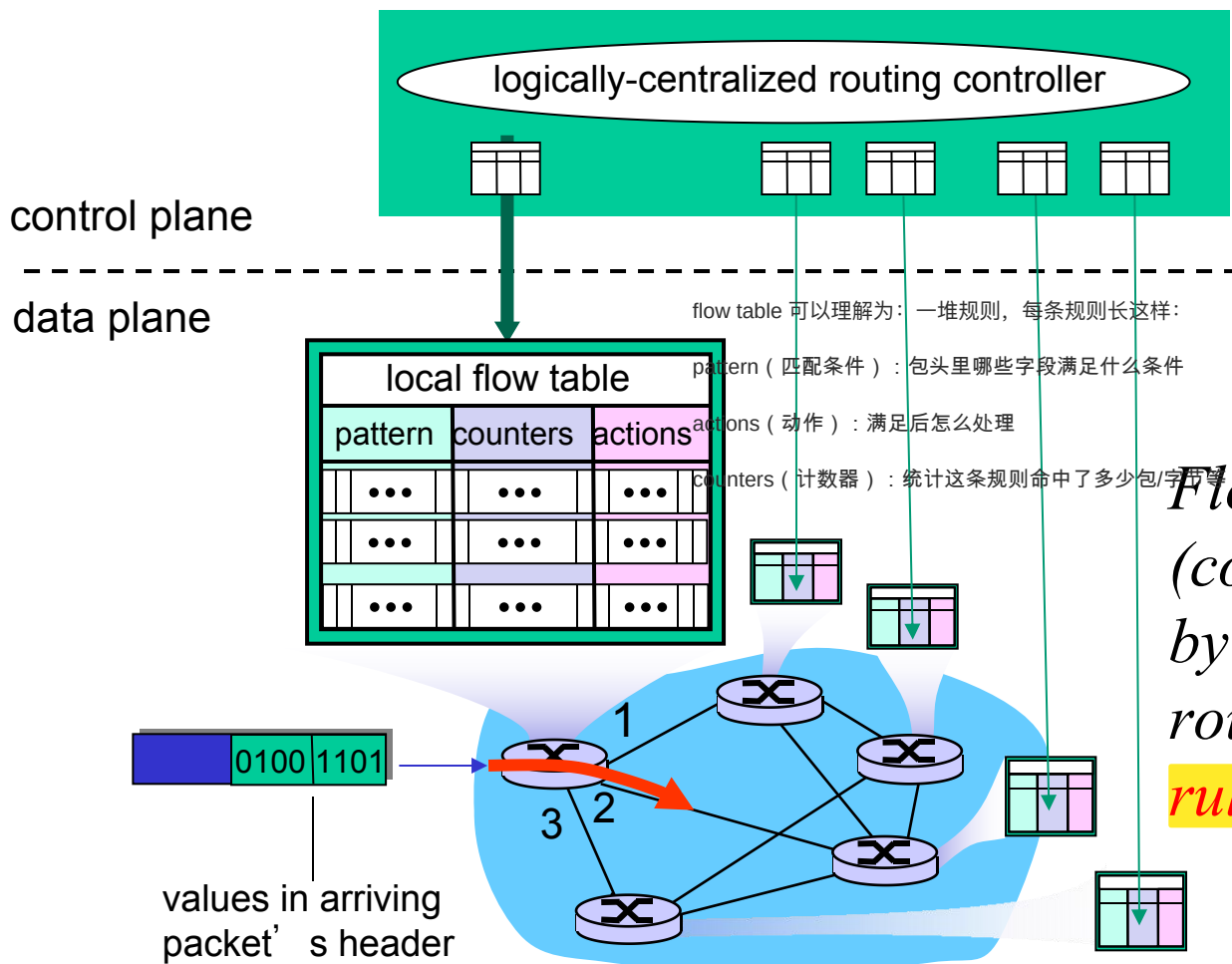
传统网络 vs SDN

传统网络：每台路由器自己跑 OSPF/BGP 等协议，自己算路由表（控制平面分布式），数据包进来就按“目的IP最长前缀匹配”转发。

SDN：把“怎么走、怎么控”交给一个（或一组）控制器统一算（控制平面更集中）；设备只负责高速执行规则（数据平面）。

*Flow table in a router
(computed and distributed
by controller) define
router's **match+action**
rules*

设备收到包后，不是“只看目的地址选路由”，而是“匹配某条规则→执行对应动作”。

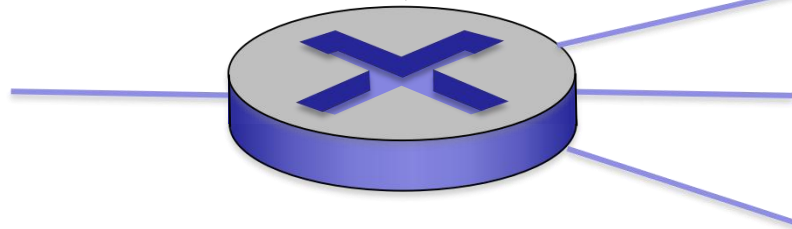


OpenFlow data plane abstraction

Generalized forwarding: simple packet-handling rules

- *Pattern*: match values in packet header fields
- *Actions*: *for matched packet*: drop, forward, modify, matched packet or send matched packet to controller
- *Priority*: disambiguate overlapping patterns
- *Counters*: #packets; time last updated

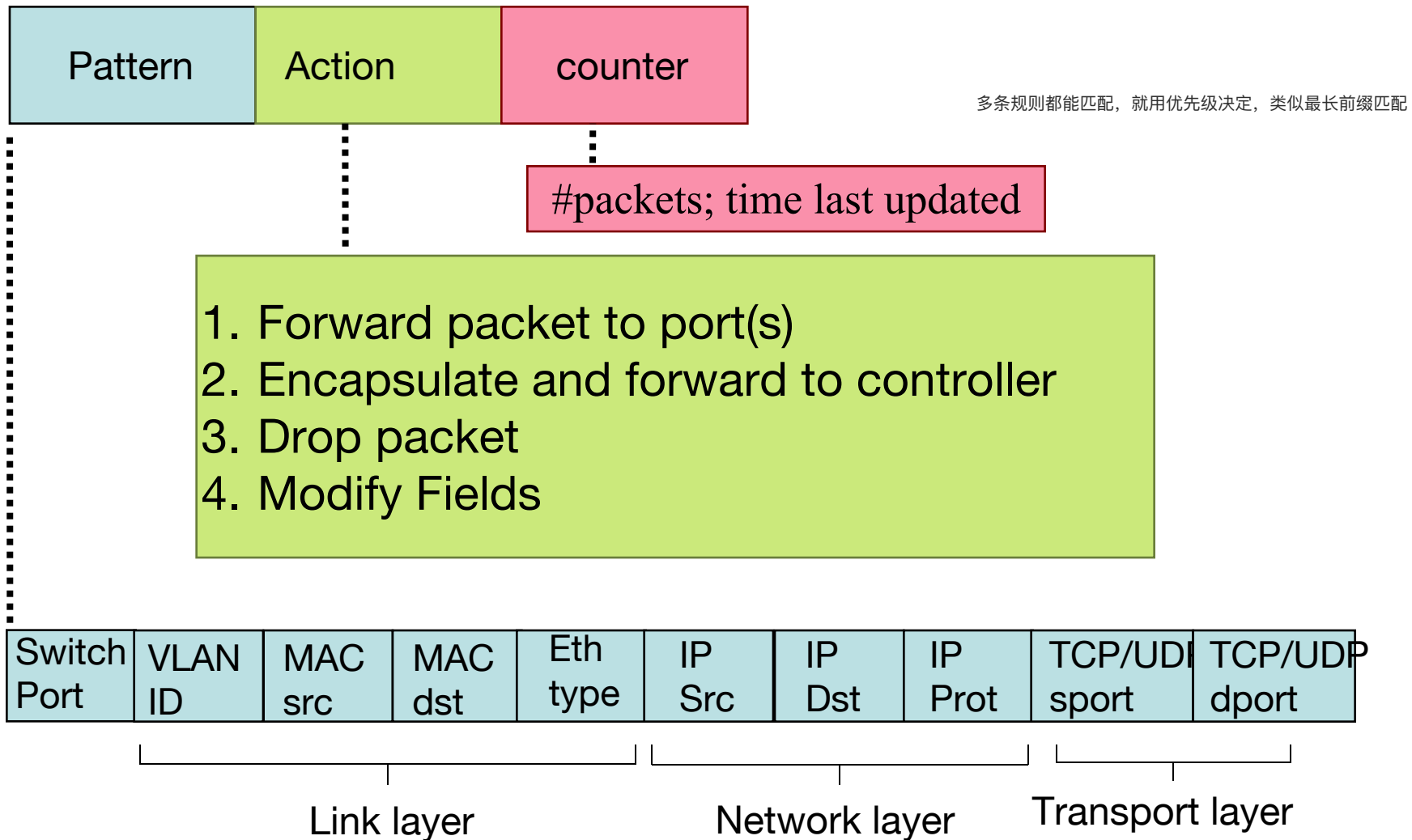
SDN/OpenFlow 把网络设备的数据平面抽象成“流表：按包头匹配 → 执行动作”，而控制器负责全局计算并下发这些规则。



1. src=1.2.*.*, dest=3.4.5.* → drop
2. src = *.*.*, dest=3.4.*.* → forward(2)
3. src=10.1.2.3, dest=.*.*.* → send to controller

从某个端口转发（像交换机/路由器）

OpenFlow: Flow Table Entries



Destination-based forwarding:

Destination-based forwarding: 传统网络设备（路由器/交换机）最经典的转发方式：基于地址的转发

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP/UDP sport	TCP/UDP dport	Action
-------------	---------	---------	----------	---------	--------	--------	---------	---------------	---------------	--------

* * * * *

51.6.0.8 * * *

IP datagrams destined to IP address 51.6.0.8 should be forwarded to router output port 6

Destination-based layer 2 (switch) forwarding:

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP/UDP sport	TCP/UDP dport	Action
-------------	---------	---------	----------	---------	--------	--------	---------	---------------	---------------	--------

* 22:A7:23:11:E1:02 *

传统 destination-based forwarding 的限制:

layer 2 frames from MAC address 22:A7:23:11:E1:02 should be forwarded to output port 6

只要 源 MAC 地址是 22:A7:23:11:E1:02
→ 转发到端口 3

这就是交换机的 MAC 地址学习/转发表模型。

不能根据 源地址

不能根据 TCP/UDP 端口

不能按“应用/用户/流”做策略

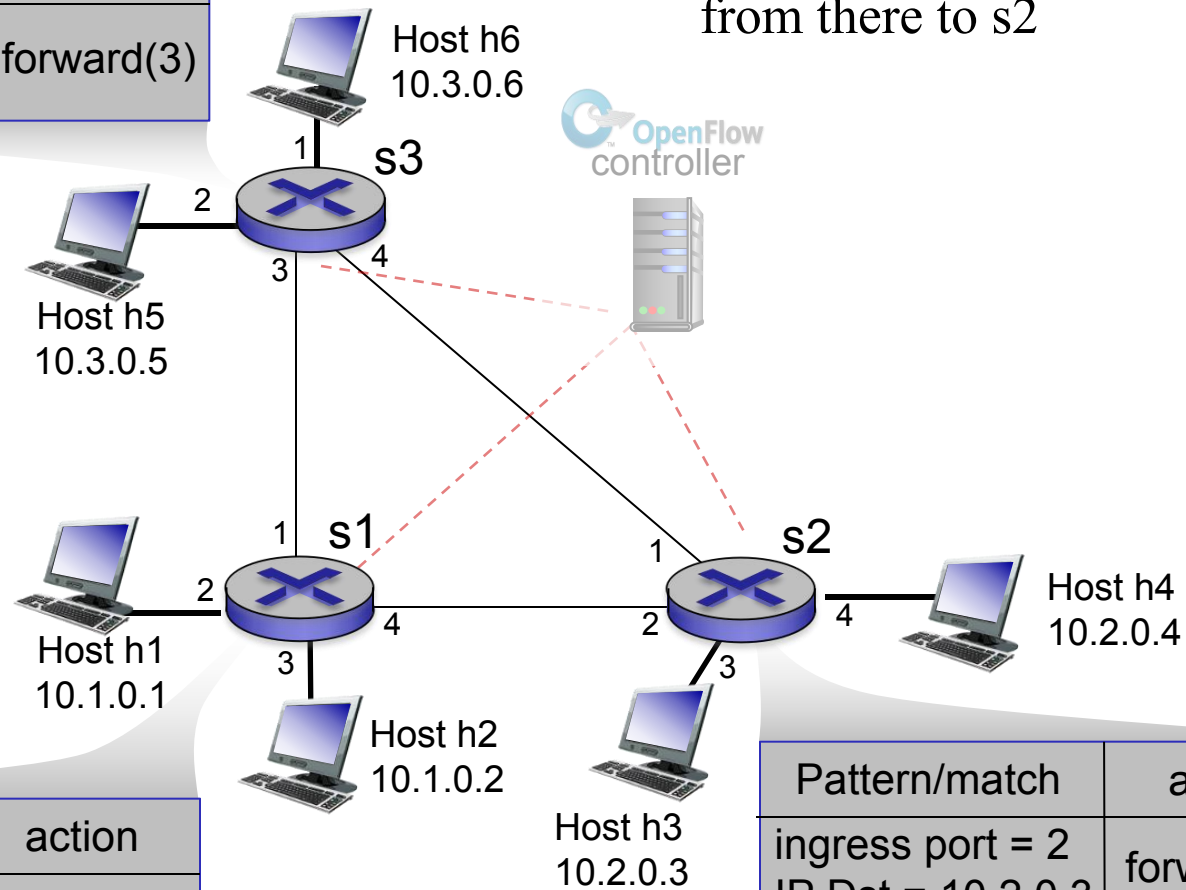
很难做精细的安全、流量工程、监控

这正是 SDN / OpenFlow 出现的原因

OpenFlow example

Example: datagrams from hosts h5 and h6 should be sent to h3 or h4, via s1 and from there to s2

Pattern/match	action
IP Src = 10.3.*.* IP Dst = 10.2.*.*	forward(3)



Pattern/match	action
ingress port = 1 IP Src = 10.3.*.* IP Dst = 10.2.*.*	forward(4)

Pattern/match	action
ingress port = 2 IP Dst = 10.2.0.3	forward(3)
ingress port = 2 IP Dst = 10.2.0.4	forward(4)

Firewall

Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP/UDP P dport	TCP/UDP P dport	Forward
----------------	------------	------------	-------------	------------	-----------	-----------	------------	-----------------------	-----------------------	---------

* * * * * * * * * 22 drop

*do not forward (block) all datagrams destined to
TCP port 22*

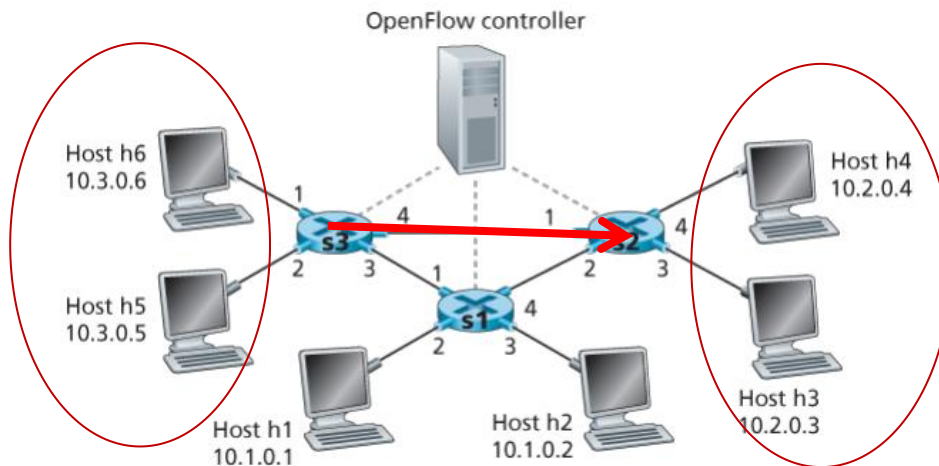
Switch Port	MAC src	MAC dst	Eth type	VLAN ID	IP Src	IP Dst	IP Prot	TCP/UDP P dport	TCP/UDP P dport	Forward
----------------	------------	------------	-------------	------------	-----------	-----------	------------	-----------------------	-----------------------	---------

* * * * * 128.119.1.1 * * * drop

*do not forward (block) all datagrams sent by host
128.119.1.1*

防火墙 = 一组“高优先级 drop 的流表规则”

Firewall



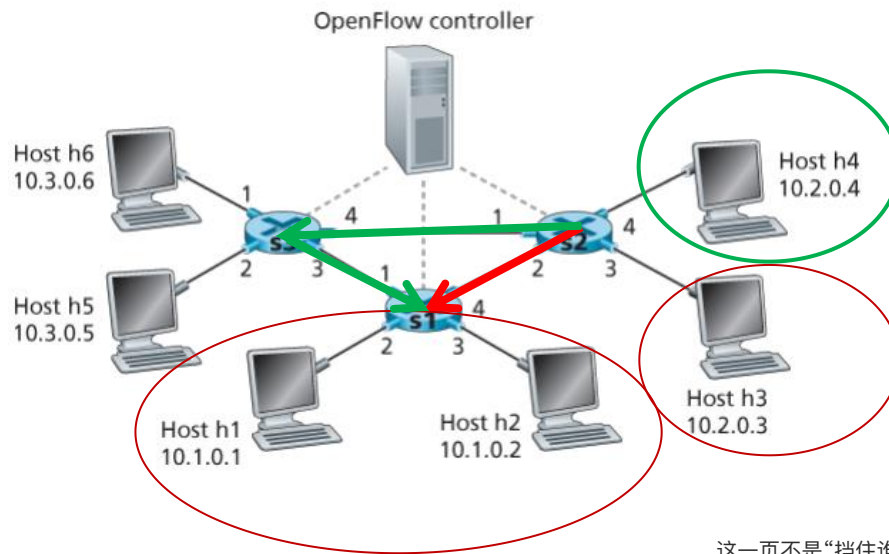
这个表算是白名单

s2 Flow Table (Example 3)	
Match	Action
IP Src = 10.3.*.* IP Dst = 10.2.0.3	Forward(3)
IP Src = 10.3.*.* IP Dst = 10.2.0.4	Forward(4)

除了上面两种流量，其他所有包在 s2 都没有匹配规则 → 默认就不转发（等价于 drop）

there were no other entries in s2's flow table

Load Balancing



这一页不是“挡住谁”，而是“把流量分摊到不同路径/端口上”，避免某一条链路或某一台机器太忙。

s2 Flow Table (Example 2)	
Match	Action
Ingress port = 3; IP Dst = 10.1.*.*	Forward(2)
Ingress port = 4; IP Dst = 10.1.*.*	Forward(1)
...	...

OpenFlow abstraction

OpenFlow 抽象把各种网络设备的“行为”统一成同一种编程模型：match + action（匹配 + 动作）。

- *match+action*: unifies different kinds of devices
- Router
 - *match*: longest destination IP prefix
 - *action*: forward out a link 按目的 IP 做匹配，然后决定从哪个接口出去
- Switch （二层交换机）
 - *match*: destination MAC address
 - *action*: forward 按目的 MAC 匹配，然后从对应端口转发
- Firewall
 - *match*: IP addresses and TCP/UDP port numbers
 - *action*: permit or deny 按五元组/端口等匹配，然后决定放行还是丢弃
- NAT
 - *match*: IP address and port
 - *action*: rewrite address and port 匹配到某个连接/流，然后改写包头字段

Chapter 4: *done!*

4.1 Overview of Network layer: data plane and control plane

4.2 What's inside a router

4.3 IP: Internet Protocol

- datagram format
- fragmentation
- IPv4 addressing
- NAT
- IPv6

4.4 Generalized Forward and SDN

- match plus action
- OpenFlow example

Question: how do forwarding tables (destination-based forwarding) or flow tables (generalized forwarding) computed?

Answer: by the **control plane** (next chapter)

Chapter 5: network layer control plane

chapter goals: understand principles behind network control plane

- traditional routing algorithms
- SDN controllers
- Internet Control Message Protocol
- network management

and their instantiation, implementation in the Internet:

- OSPF, BGP, OpenFlow, ICMP, SNMP

Chapter 5: outline

5.1 introduction

5.2 routing protocols

- link state
- distance vector

5.3 intra-AS routing in the Internet: OSPF

5.4 routing among the ISPs: BGP

5.5 The SDN control plane

5.6 ICMP: The Internet Control Message Protocol

5.7 Network management and SNMP

Network-layer functions

Recall: two network-layer functions:

- *forwarding*: move packets from router's input to appropriate router output

data plane

- *routing*: determine route taken by packets from source to destination

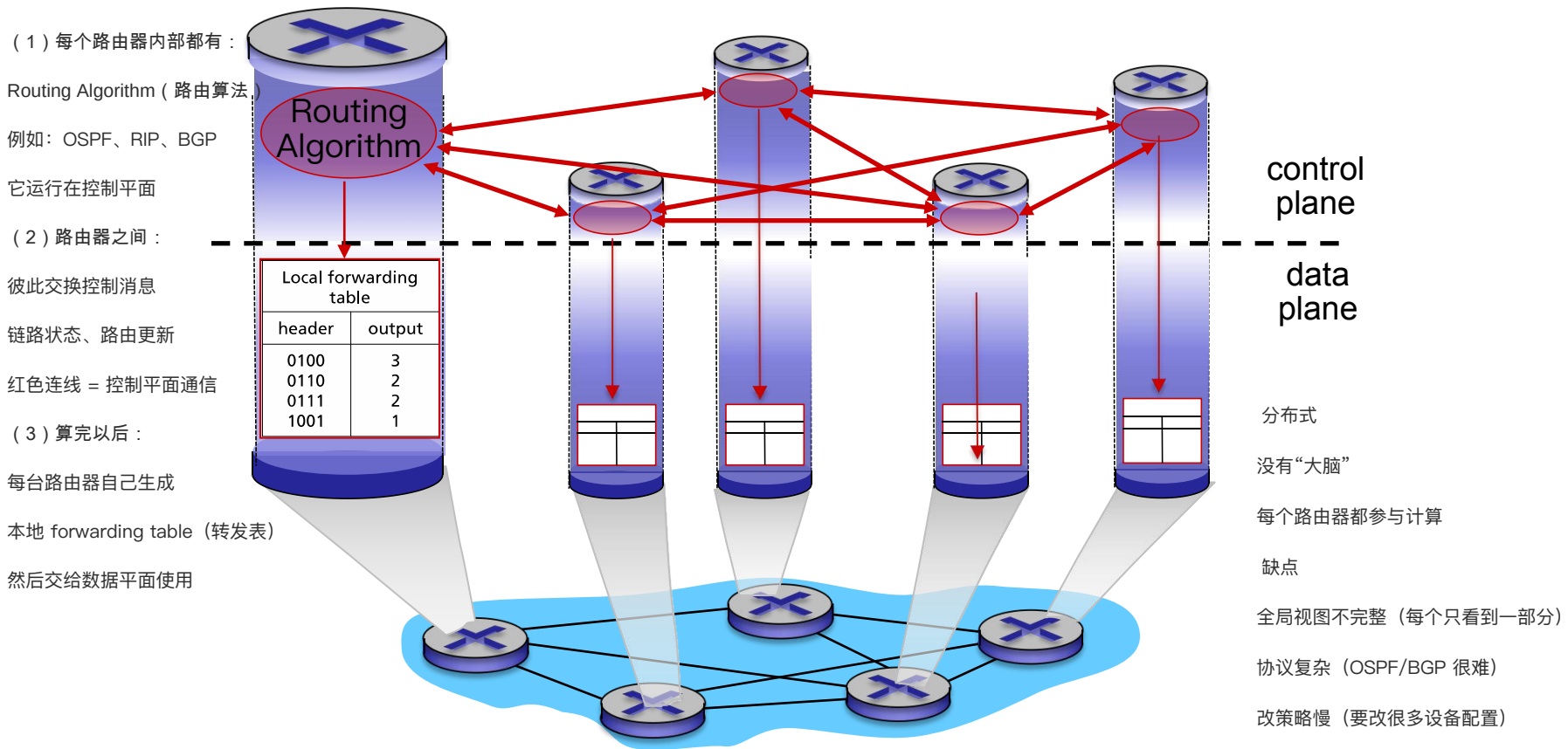
control plane

Two approaches to structuring network control plane:

- per-router control (traditional)
- logically centralized control (software defined networking)

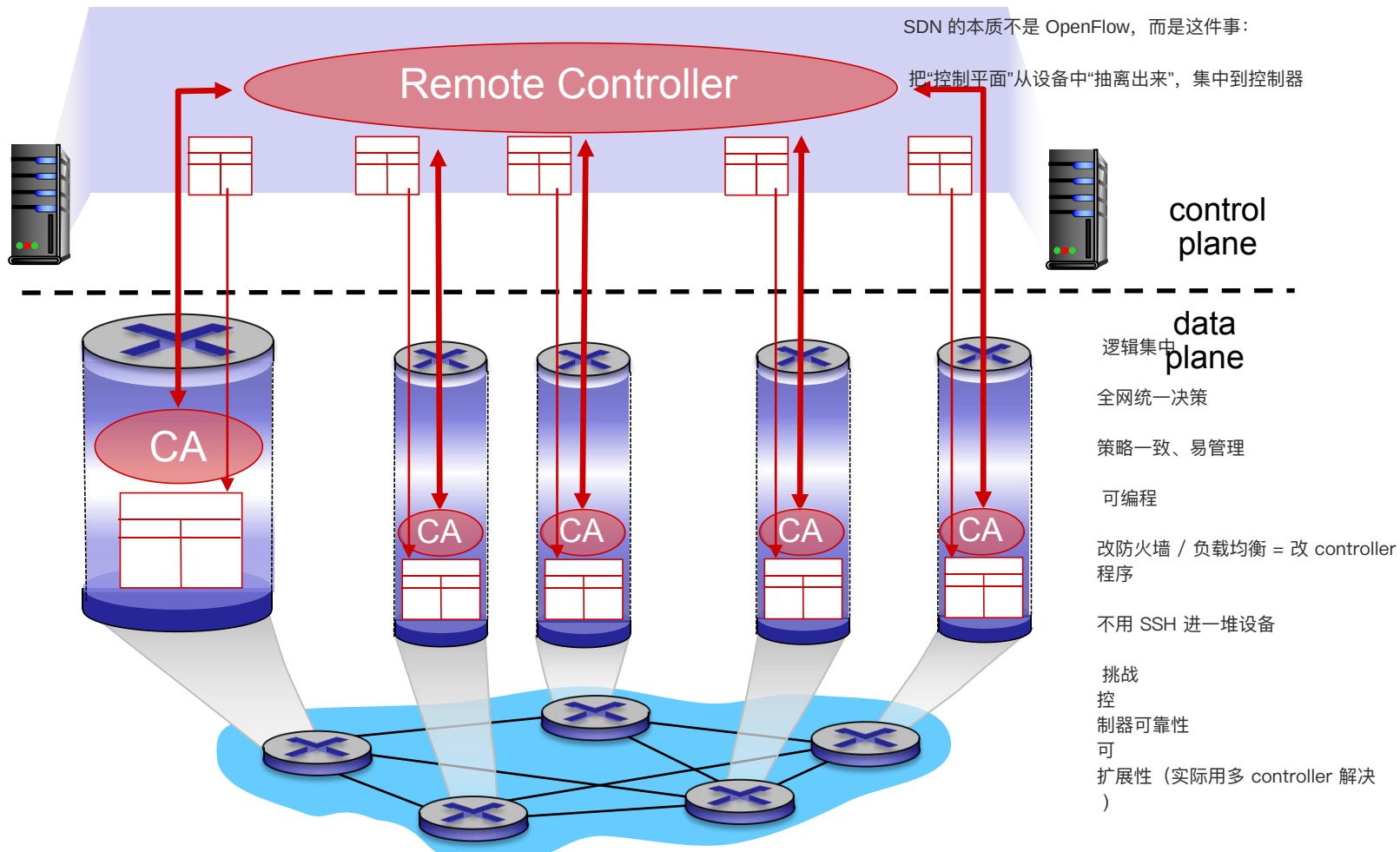
Per-router control plane

Individual routing algorithm components in each and every router interact with each other in control plane to compute forwarding tables



Logically centralized control plane

A **distinct (typically remote) controller** interacts with local control agents (CAs) in routers to compute forwarding tables



Chapter 5: outline

5.1 introduction

5.2 routing protocols

- link state
- distance vector

5.3 intra-AS routing in the Internet: OSPF

5.4 routing among the ISPs: BGP

5.5 The SDN control plane

5.6 ICMP: The Internet Control Message Protocol

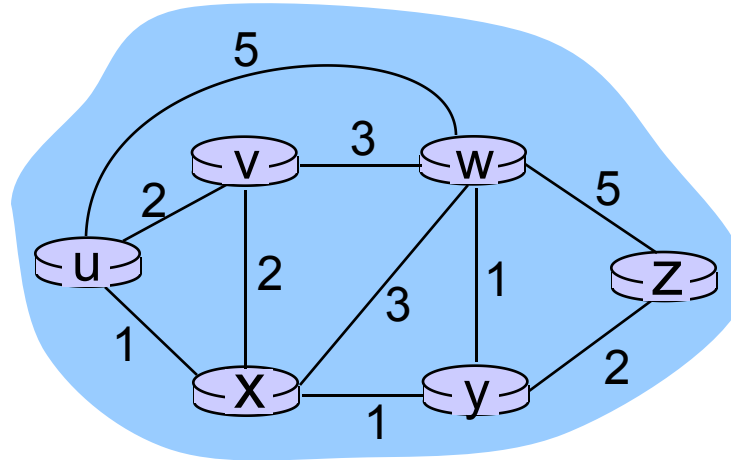
5.7 Network management and SNMP

Routing protocols

Routing protocol goal: determine “good” paths (equivalently, routes), from sending hosts to receiving host, through network of routers

- path: sequence of routers packets will traverse in going from given initial source host to given final destination host
- “good”: least “cost”, “fastest”, “least congested”
- routing: a “top-10” networking challenge!

Graph abstraction of the network



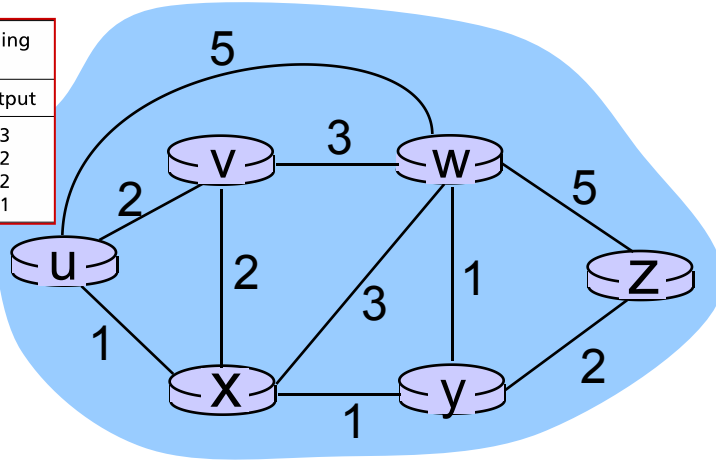
graph: $G = (N, E)$

N = set of routers = $\{ u, v, w, x, y, z \}$

E = set of links = $\{ (u,v), (u,x), (v,x), (v,w), (x,w), (x,y), (w,y), (w,z), (y,z) \}$

Graph abstraction: costs

Local forwarding table	
header	output
0100	3
0110	2
0111	2
1001	1



$c(x, x') = \text{cost of link } (x, x')$
e.g., $c(w, z) = 5$

cost could always be 1, or
inversely related to bandwidth,
or inversely related to
congestion

cost of path $(x_1, x_2, x_3, \dots, x_p) = c(x_1, x_2) + c(x_2, x_3) + \dots + c(x_{p-1}, x_p)$

key question: what is the least-cost path between u and z ?
routing algorithm: algorithm that finds that least cost path

Routing algorithm classification

Q: global or decentralized information?

global: OSPF, IS-IS

- all routers have complete topology, link cost info
- “link state” algorithms

Dijkstra

decentralized:

- router knows physically-connected neighbors, link costs to neighbors
- iterative process of computation, exchange of info with neighbors
- “distance vector” algorithms

RIP

Bellman-Ford

Q: static or dynamic?

static:

- routes change slowly over time

dynamic:

- routes change more quickly
 - periodic update
 - in response to link cost changes

Routing algorithm classification

Q: load-sensitive or load insensitive?

Load-sensitive:

- Link costs vary dynamically to reflect the current level of congestion

Load-insensitive

- A link's cost does not explicitly reflect its current level of congestion

协议	信息范围	是否动态	是否负载敏感
RIP	Decentralized	Dynamic	Load-insensitive
OSPF	Global	Dynamic	Load-insensitive
Static routing	Global (人工)	Static	Load-insensitive

Chapter 5: outline

5.1 introduction

5.2 routing protocols

- link state (global)
- distance vector (decentralized)

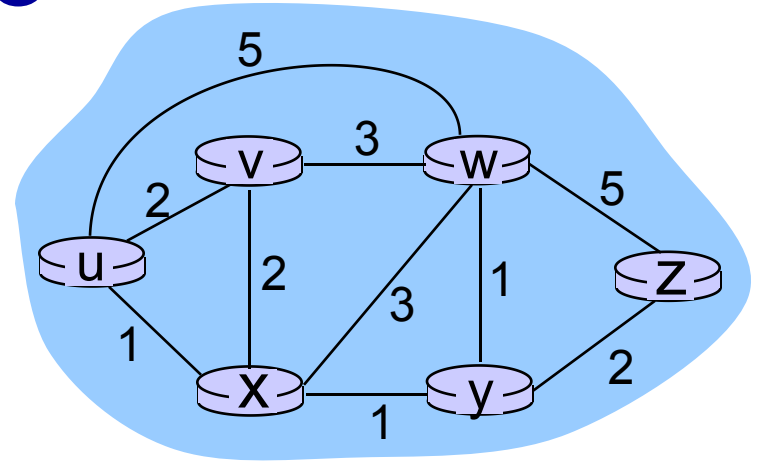
5.3 intra-AS routing in the Internet: OSPF

5.4 routing among the ISPs: BGP

5.5 The SDN control plane

5.6 ICMP: The Internet Control Message Protocol

5.7 Network management and SNMP



A link-state routing algorithm

Dijkstra's algorithm

- net topology, link costs known to all nodes
 - accomplished via “link state broadcast”
 - all nodes have same info
- computes least cost paths from one node (‘source’) **to all other nodes**
 - gives *forwarding table* for that node
- iterative: after k iterations, know least cost path to k dest.’ s

notation:

- $c(x,y)$: link cost from node x to y; $= \infty$ if not direct neighbors
- $D(v)$: current value of cost of path from source to dest. v
- $p(v)$: predecessor node along path from source to v
- N' : set of nodes whose least cost path **definitively known**

注意：进了 N' 的点，它的最短距离就不会再变了（在 Dijkstra 的前提：链路代价非负）

Dijkstra's algorithm

1 **Initialization:**

2 $N' = \{u\}$

3 for all nodes v

4 if v adjacent to u

5 then $D(v) = c(u, v)$

6 else $D(v) = \infty$

7

8 **Loop**

9 find w not in N' such that $D(w)$ is a minimum

10 add w to N'

11 update $D(v)$ for all v adjacent to w and not in N' :

12 **$D(v) = \min(D(v), D(w) + c(w, v))$**

13 /* new cost to v is either old cost to v or known

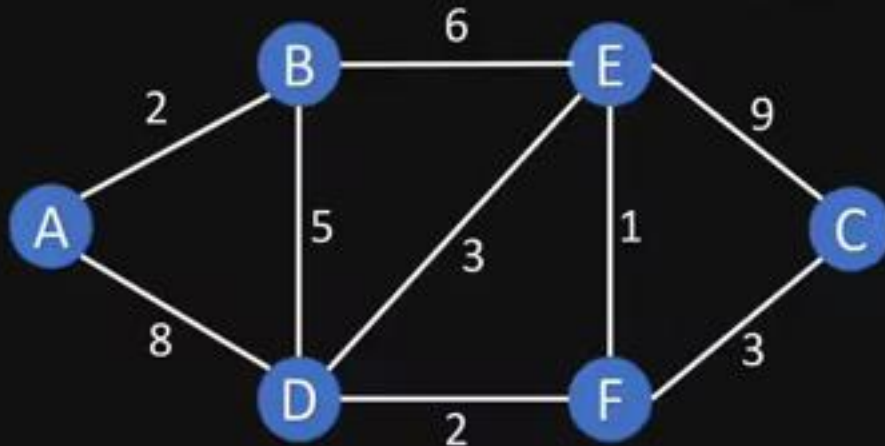
14 shortest path cost to w plus cost from w to v */

15 **until all nodes in N'**



Dijkstra's algorithm

Dijkstra's Shortest Path Algorithm



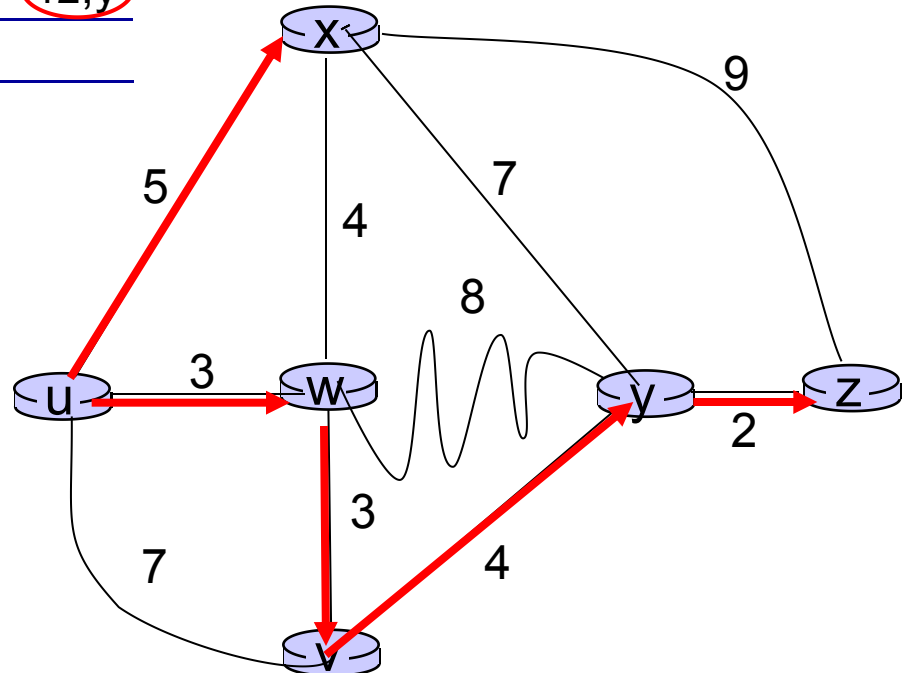
Dijkstra's Algorithm

Dijkstra's algorithm: example

Step	N'	D(v) p(v)	D(w) p(w)	D(x) p(x)	D(y) p(y)	D(z) p(z)
0	u	7,u	3,u	5,u	∞	∞
1	uw	6,w		5,u	11,w	∞
2	uwx	6,w			11,w	14,x
3	uwxv				10,v	14,x
4	uwxvy					12,y
5	uwxvyz					

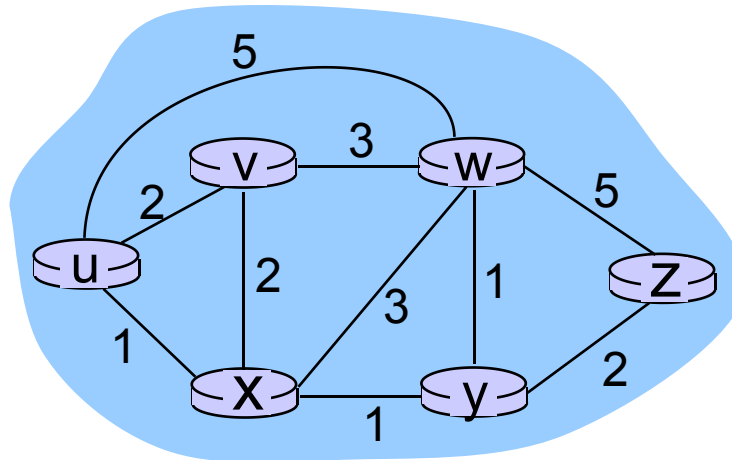
notes:

- ❖ construct shortest path tree by tracing predecessor nodes
- ❖ ties can exist (can be broken arbitrarily)



Dijkstra's algorithm: another example

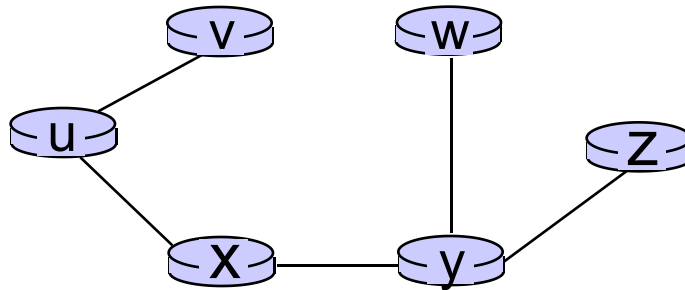
Step	N'	D(v),p(v)	D(w),p(w)	D(x),p(x)	D(y),p(y)	D(z),p(z)
0	u	2,u	5,u	1,u	∞	∞
1	ux	2,u	4,x		2,x	∞
2	uxy	2,u	3,y			4,y
3	uxyv		3,y			4,y
4	uxyvw					4,y
5	uxyvwz					



* Check out the online interactive exercises for more examples: http://gaia.cs.umass.edu/kurose_ross/interactive/

Dijkstra's algorithm: example (2)

resulting shortest-path tree from u:



resulting forwarding table in u:

destination	link
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
z	(u,x)

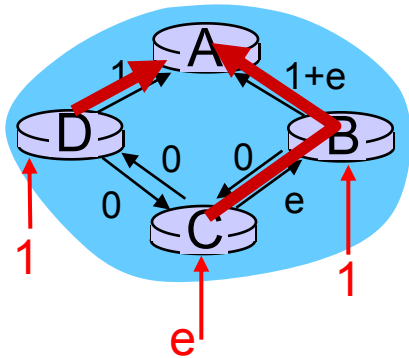
Dijkstra's algorithm, discussion

algorithm complexity: n nodes

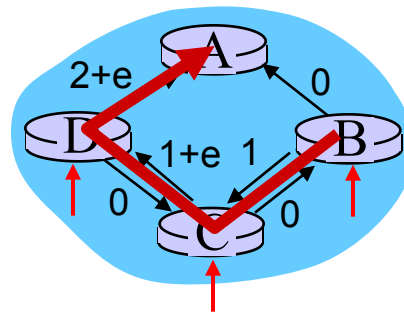
- each iteration: need to check all nodes not in N'
- $n(n+1)/2$ comparisons: $O(n^2)$
- more efficient implementations possible: $O(n \log n)$ 优先队列实现

Oscillations (摇摆) *possible (if we consider congestion)*

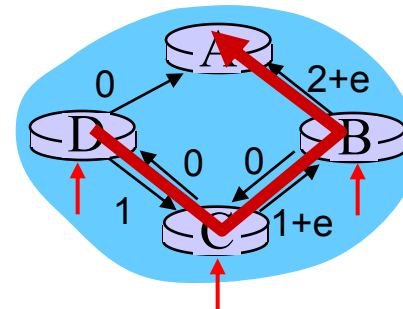
- e.g., support link cost equals amount of carried traffic:



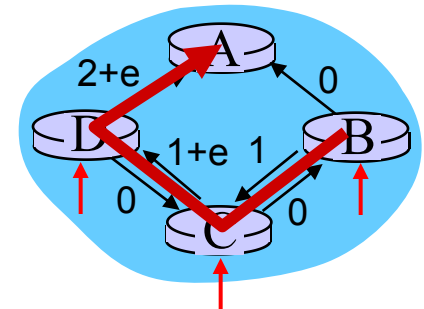
initially



given these costs,
find new routing....
resulting in new costs



given these costs,
find new routing....
resulting in new costs



given these costs,
find new routing....
resulting in new costs

Chapter 5: outline

5.1 introduction

5.2 routing protocols

- link state (global)
- distance vector (decentralized)

5.3 intra-AS routing in the Internet: OSPF

5.4 routing among the ISPs: BGP

5.5 The SDN control plane

5.6 ICMP: The Internet Control Message Protocol

5.7 Network management and SNMP

Distance vector algorithm

The distance-vector (DV) algorithm:

- **distributed**: each node **receives** some information from one or more of its directly attached neighbors, performs a **calculation**, and then **distributes** the results of its calculation back to its neighbors.
每个路由器只知道：我到邻居的链路代价 + 邻居告诉我的“它到各目的地有多远”
- **Iterative**: this process continues on until no more information is exchanged between neighbors.
当大家交换到某个状态后，发现再更新也不变了（没有更小的代价），就收敛了
- **Asynchronous**: it does not require all of the nodes to operate in lockstep with each other.

Bellman-Ford equation

Distance vector algorithm

Distance vector algorithm

Bellman-Ford equation:

$d_x(y) := \text{cost of least-cost path from } x \text{ to } y$

then

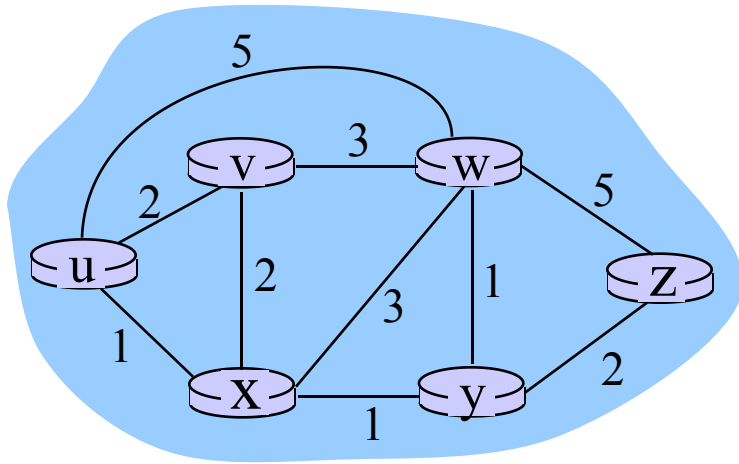
$$d_x(y) = \min_v \{ c(x,v) + d_v(y) \}$$

cost from neighbor v to destination y
cost to neighbor v

$\min$ taken over all neighbors v of x

x 要去 y，最优策略一定是：先选一个邻居 v 当下一跳，然后总代价 = “到 v 的代价” + “v 到 y 的代价”；在所有邻居里选最小的那个

Bellman-Ford example



clearly, $d_v(z) = 5$, $d_x(z) = 3$, $d_w(z) = 3$

B-F equation says:

$$\begin{aligned} d_u(z) &= \min \{ c(u,v) + d_v(z), \\ &\quad c(u,x) + d_x(z), \\ &\quad c(u,w) + d_w(z) \} \\ &= \min \{ 2 + 5, \\ &\quad 1 + 3, \\ &\quad 5 + 3 \} = 4 \end{aligned}$$

Node achieving minimum is

- the next hop in shortest path used in forwarding table

Distance vector algorithm

$D_x(y)$ = estimate of least cost from x to y

Node x:

- knows cost to each neighbor v: $c(x,v)$
- maintains its recent distance vector $\mathbf{D}_x = [D_x(y): y \in N]$
- maintains its neighbors' recent distance vectors. For each neighbor v, x maintains $\mathbf{D}_v = [D_v(y): y \in N]$

Distance vector algorithm

Key Idea:

- From time-to-time, each node sends its own recent distance vector (DV) to neighbors
- When x receives new DV from neighbor, it updates its own DV using B-F equation:

$$D_x(y) \leftarrow \min_v \{c(x,v) + D_v(y)\} \text{ for each node } y \in N$$

- If its DV has changed, sends the updated DV to neighbors

...

变化驱动的

- ❖ under minor, natural conditions, the estimate $D_x(y)$ converge to the actual least cost $d_x(y)$

在正常、小变化的情况下会逐渐收敛到真实的最短路径代价

Distance vector algorithm

iterative, asynchronous:

each local iteration caused by:

- local link cost change
- DV update message from neighbor

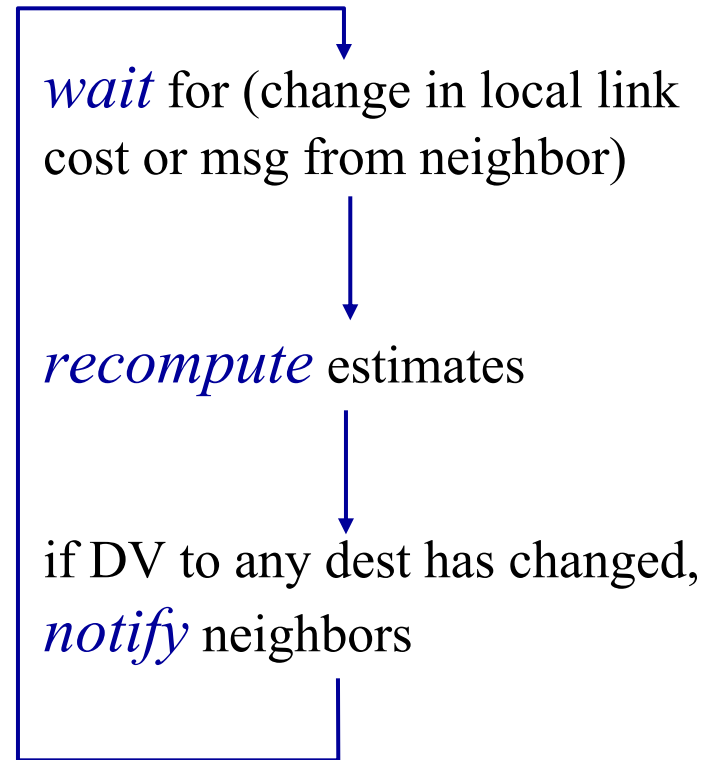
本地链路代价变化 (比如
链路拥塞、断链)

收到邻居发来的 DV 更新

distributed:

- each node notifies neighbors only when its DV changes
 - neighbors then notify their neighbors if necessary

each node:



Distance vector algorithm

```
1  Initialization:
2    for all destinations y in N:
3       $D_x(y) = c(x, y)$  /* if y is not a neighbor then  $c(x, y) = \infty$  */
4    for each neighbor w
5       $D_w(y) = ?$  for all destinations y in N
6    for each neighbor w
7      send distance vector  $D_x = [D_x(y) : y \text{ in } N]$  to w
8
9  loop
10    wait (until I see a link cost change to some neighbor w or
11          until I receive a distance vector from some neighbor w)
12
13    for each y in N:
14       $D_x(y) = \min_v \{c(x, v) + D_v(y)\}$ 
15
16    if  $D_x(y)$  changed for any destination y
17      send distance vector  $D_x = [D_x(y) : y \text{ in } N]$  to all neighbors
18
19  forever
```

它就是把上面三条规则落到程序里:

- 先把 $N' = \{x\}$ (有些讲法会用类似集合表示“已知自己”)
- 对所有目的地 y: 设置 $D_x(y) = c(x, y)$, 如果没有直连则视为 ∞
- 然后 把这个初始 DV 发给每个邻居 (“开局广播一次我目前知道的距离”)

$$D_x(y) = \min\{c(x,y) + D_y(y), c(x,z) + D_z(y)\} \\ = \min\{2+0, 7+1\} = 2$$

$$D_x(z) = \min\{c(x,y) + D_y(z), c(x,z) + D_z(z)\} \\ = \min\{2+1, 7+0\} = 3$$

**node x
table**

		cost to		
		x	y	z
from	x	0	2	7
	y	∞	∞	∞
	z	∞	∞	∞

**node y
table**

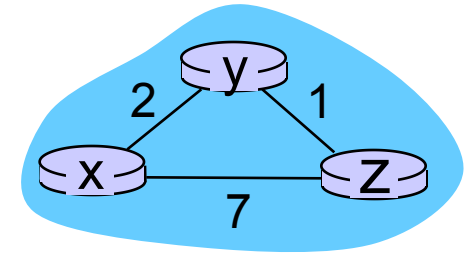
		cost to		
		x	y	z
from	x	∞	∞	∞
	y	2	0	1
	z	∞	∞	∞

**node z
table**

		cost to		
		x	y	z
from	x	∞	∞	∞
	y	∞	∞	∞
	z	7	1	0

		cost to		
		x	y	z
from	x	0	2	3
	y	2	0	1
	z	7	1	0

利用邻居信息更新x的距离向量



time

$$D_x(y) = \min\{c(x,y) + D_y(y), c(x,z) + D_z(y)\}$$

$$= \min\{2+0, 7+1\} = 2$$

$$D_x(z) = \min\{c(x,y) + D_y(z), c(x,z) + D_z(z)\}$$

$$= \min\{2+1, 7+0\} = 3$$

这一列是并行的

**node x
table**

		cost to		
		x	y	z
from	x	0	2	7
	y	∞	∞	∞
	z	∞	∞	∞

**node y
table**

		cost to		
		x	y	z
from	x	∞	∞	∞
	y	2	0	1
	z	∞	∞	∞

**node z
table**

		cost to		
		x	y	z
from	x	∞	∞	∞
	y	∞	∞	∞
	z	7	1	0

		cost to		
		x	y	z
from	x	0	2	3
	y	2	0	1
	z	7	1	0

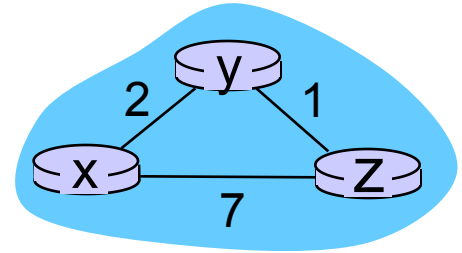
		cost to		
		x	y	z
from	x	0	2	7
	y	2	0	1
	z	7	1	0

		cost to		
		x	y	z
from	x	0	2	7
	y	2	0	1
	z	3	1	0

		cost to		
		x	y	z
from	x	0	2	3
	y	2	0	1
	z	3	1	0

		cost to		
		x	y	z
from	x	0	2	3
	y	2	0	1
	z	3	1	0

		cost to		
		x	y	z
from	x	0	2	3
	y	2	0	1
	z	3	1	0

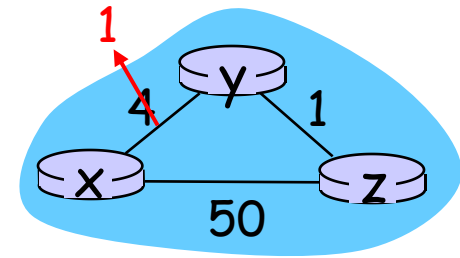


time →

Distance vector: link cost changes

link cost changes:

- ❖ node detects local link cost change
- ❖ updates routing info, recalculates distance vector
- ❖ if DV changes, notify neighbors



“good
news
travels
fast”

t_0 : y detects link-cost change, updates its DV, informs its neighbors.

t_1 : z receives update from y , updates its table, computes new least cost to x , sends its neighbors its DV.

t_2 : y receives z 's update, updates its distance table. y 's least costs do *not* change, so y does *not* send a message to z .

Distance vector: link cost changes

先从别人的拿最新的别人距离向量，然后只会对自己的行进行最新更新

**node x
table**

		cost to		
		x	y	z
from	x	0	4 1	5 2
	y	4	0	1
	z	5	1	0

Detect $c(x,y)=c(y,x)=1$!

**node y
table**

		cost to		
		x	y	z
from	x	0	4	5
	y	4 1	0	1
	z	5	1	0

**node z
table**

		cost to		
		x	y	z
from	x	0	4	5
	y	4	0	1
	z	5 1	1	0

		cost to		
		x	y	z
from	x	0	1	2
	y	1	0	1
	z	5	1	0

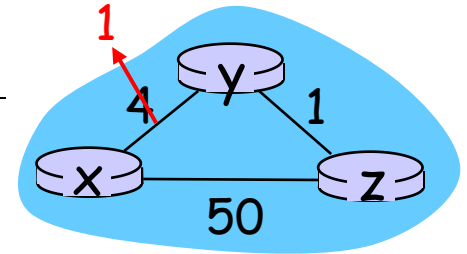
		cost to		
		x	y	z
from	x	0	1	2
	y	1	0	1
	z	5	1	0

		cost to		
		x	y	z
from	x	0	1	2
	y	1	0	1
	z	2	1	0

		cost to		
		x	y	z
from	x	0	1	2
	y	1	0	1
	z	2	1	0

		cost to		
		x	y	z
from	x	0	1	2
	y	1	0	1
	z	2	1	0

		cost to		
		x	y	z
from	x	0	1	2
	y	1	0	1
	z	2	1	0



$$D_x(z) = \min\{c(x,y) + D_y(z), c(x,z) + D_z(z)\}$$

“good news travels fast”

time

$$Dy(x) = \min\{c(y,x) + Dx(x), c(y,z) + Dz(x)\}$$

$$Dz(x) = \min\{c(z,x) + Dx(x), c(z,y) + Dy(x)\}$$

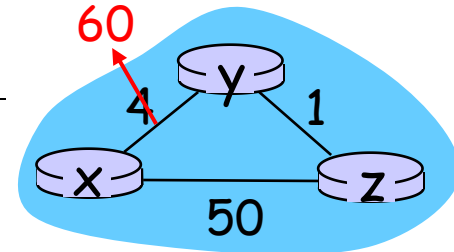
Distance Vector (DV) 路由的经典问题：count-to-infinity (“数到无穷”) / 坏消息传播很慢

**node x
table**

		cost to		
		x	y	z
from	x	0	4 51	5 50
	y	4	0	1
	z	5	1	0

		cost to		
		x	y	z
from	x			
	y			
	z			

		cost to		
		x	y	z
from	x			
	y			
	z			



Detect $c(x,y) = c(y,x) = 60$!

**node y
table**

		cost to		
		x	y	z
from	x	0	4	5
	y	4 6	0	1
	z	5	1	0

		cost to		
		x	y	z
from	x	0	51	50
	y	6	0	1
	z	5	1	0

		cost to		
		x	y	z
from	x	0	51	50
	y	8	0	1
	z	7	1	0

**node z
table**

		cost to		
		x	y	z
from	x	0	4	5
	y	4	0	1
	z	5 1	1	0

		cost to		
		x	y	z
from	x	0	51	50
	y	6	0	1
	z	7	1	0

		cost to		
		x	y	z
from	x			
	y			
	z			

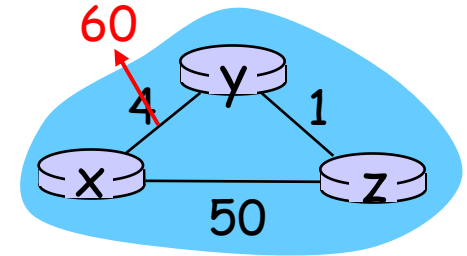
time

loop will persist for 44 iterations until z eventually computes the cost of its path via y to be greater than 50.

Distance vector: link cost changes

link cost changes:

- ❖ node detects local link cost change
- ❖ *bad news travels slow* - “count to infinity” problem!
- ❖ 44 iterations before algorithm stabilizes:
 - ❖ $D_y(x) = \min\{c(y,x) + D_x(x), c(y,z) + D_z(x)\} = \min\{60 + 0, 1 + 5\} = 6$
 - ❖ $D_z(x) = \min\{c(z,x) + D_x(x), c(z,y) + D_y(x)\} = \min\{50 + 0, 1 + 6\} = 7$
 - ❖ $D_y(x) = 8, D_z(x) = 9, \dots$ totally 44 iteration!



Poisoned reverse:

- ❖ If Z routes through Y to get to X :
 - Z tells Y its (Z's) distance to X is infinite (so Y won't route to X via Z)
- ❖ will this completely solve count to infinity problem?

poisoned reverse 只能阻止“我经你到目的地”这种两节点回路。

但如果是 3 个及以上节点形成环 (A 以为经 B, B 以为经 C, C 以为经 A), poisoned reverse 就不够用了, 坏消息仍然可能沿着更长的环慢慢“数到无穷”

$$Dy(x) = \min\{c(y,x) + Dx(x), c(y,z) + Dz(x)\}$$

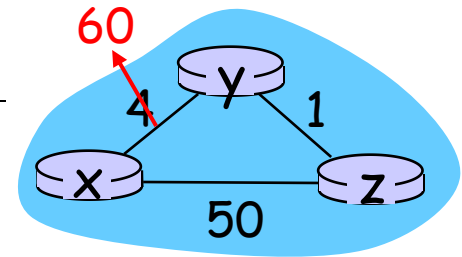
$$Dz(x) = \min\{c(z,x) + Dx(x), c(z,y) + Dy(x)\}$$

**node x
table**

		cost to		
		x	y	z
from	x	0	4 51	50
	y	4	0	1
	z	5	1	0

		cost to		
		x	y	z
from	x			
	y			
	z			

		cost to		
		x	y	z
from	x			
	y			
	z			



Detect $c(x,y) = c(y,x) = 60$!

**node y
table**

		cost to		
		x	y	z
from	x	0	4	∞
	y	4 6	0	1
	z	0	1	0

		cost to		
		x	y	z
from	x	0	51	50
	y	60	0	1
	z	∞	1	0

		cost to		
		x	y	z
from	x	0	51	50
	y	51	0	1
	z	50	1	0

**node z
table**

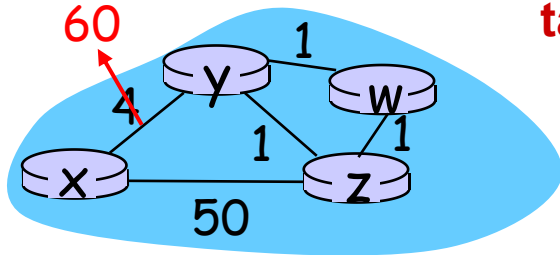
		cost to		
		x	y	z
from	x	0	4	5
	y	4	0	1
	z	5	1	0

		cost to		
		x	y	z
from	x	0	51	50
	y	60	0	1
	z	50	1	0

		cost to		
		x	y	z
from	x			
	y			
	z			

time

Distance vector: link cost changes



node x table

	cost to				
	x	y	z	w	
from	x	0	4	5	5
y	4	0	1	1	
z	5	1	0	1	

node y table

	cost to				
	x	y	z	w	
from	x	0	4	∞	∞
y	4	0	1	1	
z	4 6	1	0	1	
w	0	1	1	0	

node z table

	cost to				
	x	y	z	w	
from	x	0	4	5	5
y	4	0	1	1	
z	5	1	0	1	
w	5	1	1	0	

node z table

	cost to				
	x	y	z	w	
from	x	0	4	5	5
y	60	0	1	1	
z	6	1	0	1	
w	5	1	1	0	

node w table

	cost to				
	x	y	z	w	
from	y	60	0	1	1
z	5	1	0	1	
w	6	1	1	0	

node w table

	cost to				
	x	y	z	w	
from	y	4	0	1	1
z	5	1	0	1	
w	5	1	1	0	

node y table

	cost to				
	x	y	z	w	
from	x	0	4	∞	∞
y	7	0	1	1	
z	6	1	0	1	
w	6	1	1	0	

Poisoned reverse:

- ❖ will this completely solve count to infinity problem?
- ❖ No, when the loops involves three or more nodes

Comparison of LS and DV algorithms

message complexity

- **LS:** with n nodes, E links, $O(nE)$ msgs sent
- **DV:** exchange between neighbors only
 - convergence time varies

LS (Link State , 链路状态) : 每个路由器把“我跟邻居的链路代价”广播给全网; 大家都知道整张拓扑, 然后各自跑 Dijkstra。

DV (Distance Vector , 距离向量) : 只跟邻居交换“我到各个目的地的最短距离估计”; 用 Bellman-Ford 迭代收敛

speed of convergence

- **LS:** $O(n^2)$ algorithm requires $O(nE)$ msgs
- **DV:** convergence time varies
 - may be routing loops
 - count-to-infinity problem

robustness: what happens if router malfunctions?

LS:

- node can advertise incorrect *link* cost
- each node computes only its *own* table

DV:

- DV node can advertise incorrect *path* cost
- each node's table used by others
 - error propagate thru network

Chapter 5: outline

5.1 introduction

5.2 routing protocols

- link state
- distance vector

5.3 intra-AS routing in the Internet: OSPF

5.4 routing among the ISPs: BGP

5.5 The SDN control plane

5.6 ICMP: The Internet Control Message Protocol

5.7 Network management and SNMP